

Project 05 — Dual-Core RV32I SoC with Simplified Coherent Memory

0. What this document is

A frank, expert read-through of the project brief plus a concrete module spec and day-by-day plan, locked to the chosen toolchain. No fluff — the goal is that after reading this you understand the scope precisely and can execute without surprises.

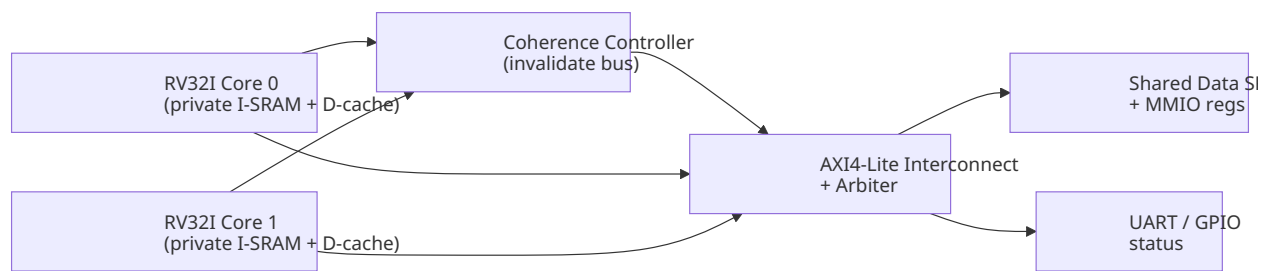
1. The one-sentence essence

Build **two** simple RV32I cores that share one data memory through a **simplified invalidation-based coherence mechanism**, wrap it on an **AXI4-Lite bus**, simulate it (directed + UVM), synthesize it, run it through **OpenLane physical design on sky130A**, and boot it on an **FPGA** that shows coherence events on UART/LEDs.

2. Locked-in decisions (toolchain confirmed)

#	Decision	Choice	Rationale
1	PDK	sky130A	Assigned. Standard cells sky130_fd_sc_hd (high-density) recommended for digital.
2	Physical flow	OpenLane (dockerized libreLane) → Yosys synth + OpenROAD PD	Assigned. Fully open-source, reproducible.
3	Bus	AXI4-Lite	Course-required. 5 independent channels (AW/W/B/AR/R); learn the handshake, don't fear it.
4	Core style	Single-cycle RV32I	Predictable timing, easiest to debug, fastest to close timing on sky130 (~25–50 MHz realistic).
5	Write policy	Write-through	Simplest correctness; no dirty-writeback logic.
6	Coherence states	I / S / M (3 states)	Minimal mechanism that demonstrates the concept. Not MESI.
7	Cache	Direct-mapped, 2–4 lines, 1 word/line	Tiny is fine; concept demonstration, not performance.
8	Reset	Single async active-low rst_n, synchronized release	One policy, used everywhere.
9	FPGA	Spec both Arty A7-100T & Zybo (decide later)	See §13.

3. The four pillars



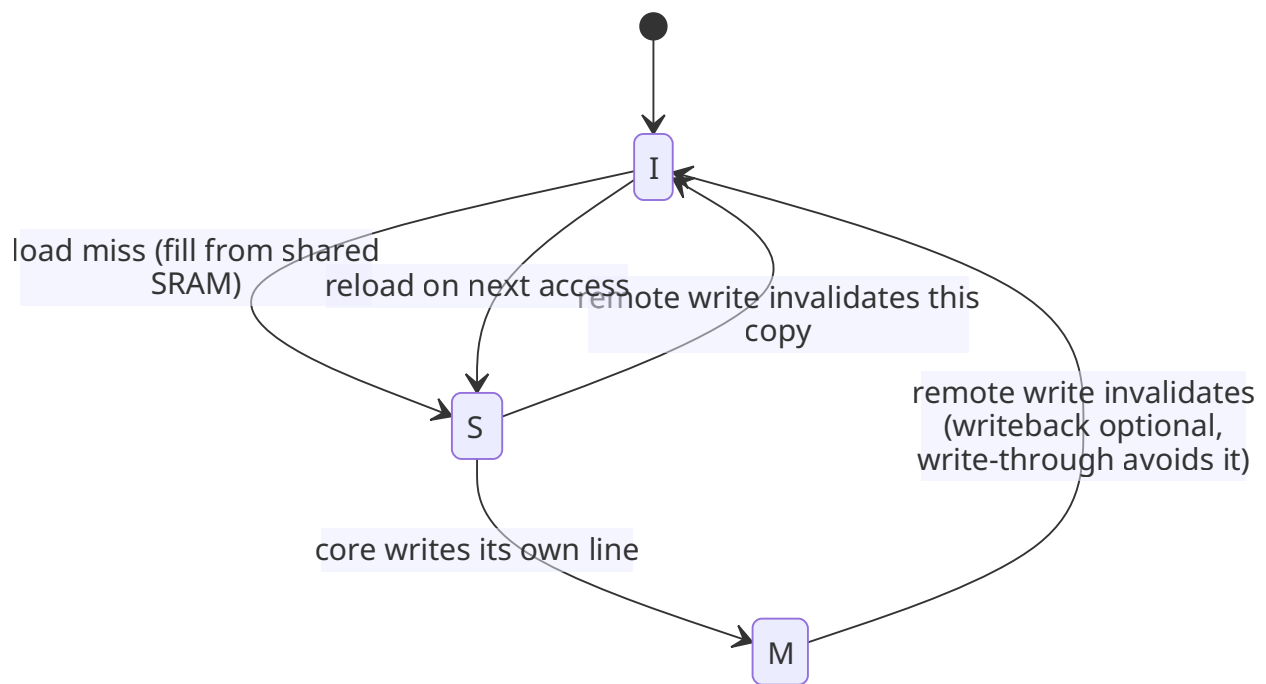
1. **Two RV32I cores** — the compute.
2. **Private caches + coherence controller** — the conceptual heart.
3. **Shared SRAM + bus arbiter** — the single source of truth.
4. **Verification + physical design + FPGA** — proof it works in silicon and hardware.

4. Module-by-module — what is actually being graded

Module	What it really is	Hidden difficulty
Two RV32I cores	32-bit base integer ISA, single-cycle.	Must be synthesizable and cycle-clean; bugs here poison everything downstream.
Instruction SRAMs	Private per-core, preloaded with each core's program.	Trivial in sim, needs init (\$readmemh /BRAM init) for FPGA & OpenRAM for ASIC.
Private data cache	Small (2–4 lines). Tag + data + state per line.	The "state" field is where coherence lives.
Cache tag/data storage	Storage backing the cache.	Keep it small.
Coherence controller	Drives invalidation on remote write; tracks line states.	The intellectual core. Graded on <i>correctness of the demo</i> , not protocol completeness.
Invalidation logic	When core A writes a shared line, core B's copy is marked stale.	Must actually force a re-fetch, not just flip a bit nobody reads.
Shared data SRAM	One memory, one truth.	Single-port arbitration timing.
Arbiter / interconnect	Serialize two masters to one SRAM.	Round-robin is fine; fairness + deadlock-free matters.
AXI4-Lite	Bus fabric + control interface.	5 channels add verbosity; handshake bugs are common.
Coherence status/debug regs	MMIO-visible state for SW/debug.	Cheap, big leverage on the FPGA demo.
UART / GPIO	Human-visible proof.	UART is the most reliable "it works" signal on FPGA.

5. The coherence model (what they actually want)

They explicitly say: **"A complete industry-level coherence protocol is not required."** Do NOT implement MESI/MOESI snooping. Build the minimal mechanism:



Three states: **I** (Invalid), **S** (Shared/clean), **M** (Modified, this core owns it). On a write by core A:

- Core A → line **M**, writes shared SRAM (write-through).
- Coherence controller signals core B: "invalidate line X."
- Core B marks its copy **I** → next read refetches the updated value.

That single flow is the entire "demonstrate coherence" requirement.

6. What the brief deliberately excludes

"Pipelining, advanced cache hierarchies, and scalable multi-core coherence protocols are **not required**."

- **No pipeline** — single-cycle core. Don't build a 5-stage pipeline.
- **No L2 / multi-level cache**. One tiny private cache per core.
- **No scalable snooping/directory bus**. Two cores, hardwired invalidate.
- **No interrupts/exceptions/privileged CSRs** beyond debug regs. RV32I base has no CSR instr.
- **No DRAM controller** — SRAM only.

Effort spent on excluded items is **wasted**.

7. The three verification layers

Layer	Tool	Purpose	What "done" looks like
Basic	Directed self-checking SV TB	Prove the coherence story end-to-end.	Write from core 0, read from core 1, assert new value appears <i>after</i> invalidation. + reset + conflict cases.
UVM	Agents, seqs, scoreboard, coverage	Randomized stress of concurrent accesses.	Scoreboard predicts shared-mem state; coverage hits hit/miss, invalidation, simultaneous-write, errors.

Layer	Tool	Purpose	What "done" looks like
FPGA	Hardware	Visible proof on LEDs/ UART.	LED blinks per core/event; UART prints before/after shared value.

These are **three separate deliverables**. The directed TB is your fallback truth.

8. Physical design on sky130A / OpenLane — the part most students underestimate

OpenLane flow = **Yosys synthesis** → **OpenROAD floorplan** → **powerplan** → **placement** → **CTS** → **routing** → **sign-off (DRC/KLayout LVS)**. Concrete reports you must produce:

- **SDC** — clock definition + I/O delays. Realistic sky130 target: **start at 20 MHz (50 ns period)**, tighten toward 40–50 MHz if timing closes. Don't chase 100 MHz in sky130.
- **Floorplan** → core utilization ~50–60% (leave room for routing on a small design).
- **Power plan** → standard ring + stripes; sky130 metal stack (M1–M5/M6).
- **Placement** → **CTS** → **routing**.
- **Reports required:** area, timing (setup/hold WNS/TNS), power, congestion, **DRC clean**, **LVS clean**, final layout screenshots.

sky130 / OpenLane-specific gotchas:

- **SRAM macros:** OpenLane supports **OpenRAM** for generating SRAMs. For tiny sizes (shared data SRAM, I-SRAMs) you can either (a) generate an OpenRAM macro, or (b) use a **register-array / flip-flop-based memory** which is fully synthesizable and avoids macro/integration pain. For a 5-day project, **recommend register-array for I-SRAM (small, e.g. 1–4 KB) and a small OpenRAM macro OR register array for shared data SRAM**. Register arrays close timing fine at 20–40 MHz.
- **Don't infer latches** — Yosys will warn; OpenROAD sign-off DRC may flag them.
- **Async reset** — infer properly (`always @(posedge clk or negedge rst_n)`); sky130 sc_hd has proper reset flops.
- **Black-box RAMs** — if using OpenRAM macros, you must provide a Liberty/LEF for them to OpenLane; if using register arrays, no black-boxing needed.
- **Run a synthesis smoke test as soon as top RTL exists** (day 2–3) so day 5 isn't the first time tools see your design.

9. FPGA demo — what "demonstrate" concretely means

The grader wants to see coherence happen:

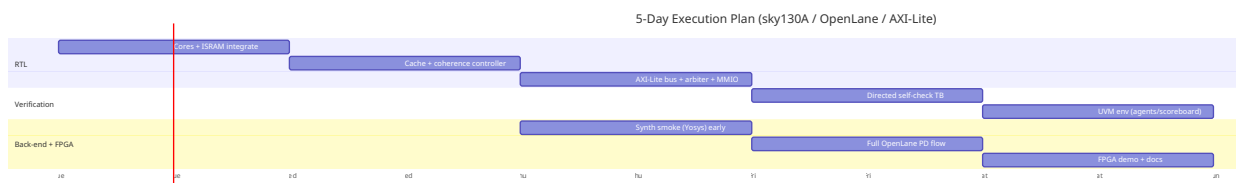
- Core 0 writes a known value to shared addr X (LED/UART: "C0 write X").
- Coherence event fires (LED pulse / UART: "INVALIDATE → C1").
- Core 1 reads X and gets the **new** value (LED/UART shows it).
- A stale read fails the demo — so it doubles as a live self-check.

UART > LEDs alone — UART logs the sequence. Use both: LEDs for heartbeat/event, UART for values.

10. Toolchain-specific decisions to make on day 1

1. **SRAM strategy:** OpenRAM macros vs register arrays. (*Recommendation: register arrays for I-SRAM; small shared data as register array OR a tiny OpenRAM macro. Register arrays = zero macro-integration pain in OpenLane.*)
 2. **Clock target:** start SDC at 20 MHz, tighten if closes.
 3. **Core clock for FPGA:** use board reference clock (Arty 100 MHz, Zybo 125/100 MHz) through an MMCM/PLL to generate a safe core clock (**50 MHz** recommended) — don't run the RV32I at the raw board clock.
 4. **FPGA BRAM** for I-SRAMs and shared data on FPGA (instantiate as block RAM, init from `.mem` / `.mif`).
-

11. Realistic 5-day plan (risk-weighted)



Risk callouts:

- **Day 1 (cores + ISRAM)** — highest leverage. *Mitigation:* use a known-good single-cycle core; don't reinvent.
- **Day 2 (cache + coherence)** — small mechanism. *Mitigation:* 3-state I/S/M, write-through, hardwired invalidate.
- **Day 3 (AXI-Lite + MMIO)** — handshake bugs common. *Mitigation:* write a tiny AXI-Lite manager/ subordinate BFM early; test channels in isolation.
- **Day 4 (directed TB + UVM)** — *Mitigation:* directed TB solid first; UVM layered on top.
- **Day 5 (full PD + FPGA + docs)** — *Mitigation:* **synth smoke test on day 3** so day 5 isn't the first tool run.

Single biggest schedule killer: **leaving synthesis until the last day**. Do a Yosys synthesis check as soon as top RTL exists.

12. Top-level signal list (anchor the design)

- `clk`, `rst_n` (async, active-low, synchronized release)
 - AXI4-Lite per master: `awvalid/awaddr/awready`, `wvalid/wdata/wstrb/wready`, `bvalid/bresp/bready`, `arvalid/araddr/arready`, `rvalid/rdata/rresp/rready`
 - MMIO region (memory-mapped): coherence status reg, event counters (invalidations/hits/misses), core-to-core doorbell, UART TX/RX regs
 - UART TX/RX pins, LED outputs
 - (FPGA) `clk_in` from board reference → MMCM/PLL → `core_clk`
-

13. FPGA board options (spec both, decide later)

Aspect	Arty A7-100T	Zybo (Zynq-7010/7020)
Chip	Artix-7 XC7A100T	Zynq-7010/7020 (PL side)
Approach	Pure FPGA (PL only) — simplest.	Use PL only , ignore ARM PS. More complex constraints.
Logic cells	~100K — ample for dual RV32I + coherence.	~28K (7010) / ~85K (7020) — 7010 is tight for dual core; 7020 fine.
BRAM	135 × 36Kb — plenty for I-SRAMs + shared.	60 (7010) / 140 (7020) × 36Kb.
Clock	100 MHz onboard ref → MMCM → 50 MHz core.	125 MHz ref → MMCM → 50 MHz core.
UART/LEDs	4 LEDs + 4 switches + Pmod; onboard USB-UART.	4 LEDs + 4 switches; onboard USB-UART.
Verilog flow	Vivado, <code>.xdc</code> constraints.	Vivado, <code>.xdc</code> (PL-only).
Recommendation	Preferred — pure PL, no Zynq PS complexity, large device.	OK if only board available; use PL-only, avoid PS.

Recommendation: **Arty A7-100T** unless the Zybo is your only option. Pure FPGA avoids Zynq PS boot/FSBL complexity that adds nothing the grader asked for.

14. The transparent warnings (what the brief doesn't say)

- **Coherence only fixes *future* loads.** If core B already has a value in a register, invalidating its cache line does nothing. The demo program must re-read *after* the invalidation. Design demo SW around this. **#1 silent bug.**
- **"Simultaneous writes"** need a deterministic arbiter winner + loser invalidation. Brief lists it as a test case.
- **Instruction fetch must NOT go through the coherence path** — only data is shared; I-SRAMs are private.
- **Reset behavior:** all caches → Invalid, all state → known. Directed reset test is required.
- **"Error conditions":** define yours (bus error/SLVERR, unmapped access) and handle them — or UVM coverage exposes the gap.
- **Coverage is a deliverable:** code coverage + functional coverage. Add coverpoints for coherence states and events.
- **AXI4-Lite is not AXI4.** No bursts, no `wlast`/`rlast`, no `awlen`/`awsize`/`awburst`. Keep it strictly Lite — 5 channels, one transaction at a time per channel. Don't over-engineer.
- **OpenLane RAM macros:** if you go OpenRAM, you must supply `.lib` + `.lef` + `.gds` to the flow and black-box in Yosys. Register arrays avoid all of this. Decide early.

15. Minimum viable demo program

```
# Core 0 # Core 1
sw r1, 0(shared) # writes X (spin) lw r2, 0(shared) # expects X after invalidation
# coherence controller invalidates C1's line first
# so this lw refetches = sees X, not stale
```

If this passes in directed sim **and** on the FPGA, the project's core thesis is demonstrated.

16. Bottom line

- **Scope:** two small RV32I cores + tiny private caches + a 3-state invalidate-on-write coherence link + one shared SRAM on AXI4-Lite, verified three ways, pushed through Yosys/OpenLane on sky130A, shown on FPGA.
- **Intellectual center of gravity:** the coherence controller + the demo proving stale data gets invalidated.
- **Schedule risk centers:** day 1 (CPU), day 5 (back-end). Mitigated by reusing a clean core + early synth smoke test.
- **The brief gives you permission to stay small.** Use it. Small + correct + fully-verified + cleanly-physical-design'd beats big + half-finished every time.

Next step: I can turn this into (a) a detailed module-by-module RTL spec with signal lists and interfaces, or (b) a concrete day-1 starter (core integration + AXI-Lite skeleton). Tell me which to build first.